

机器视觉实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

1 实验目的

1. 深入理解数字图像的基本概念与性质，掌握图像像素矩阵的访问与操作方法。
2. 掌握灰度直方图的计算原理，并能够通过 C++ 与 OpenCV 编程实现直方图的计算与可视化显示。
3. 掌握空间域图像卷积与频率域滤波的关系，理解并利用卷积定理与快速傅里叶变换（FFT）实现大规模图像和滤波核的高效卷积计算。

2 实验原理

2.1 直方图计算原理（任务二）

灰度直方图（Brightness Histogram）反映了图像中不同灰度级出现的概率或像素个数。对于一幅大小为 $M \times N$ 、灰度级范围为 $[0, 255]$ 的灰度图像 $f(m, n)$ ，其直方图 H 可以通过遍历图像所有像素来统计：

$$H[k] = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} \delta(f(m, n) - k), \quad k \in [0, 255] \quad (1)$$

其中 $\delta(x)$ 为单位冲激函数。在编程实现时，通过创建大小为 256 的数组并初始化为 0，遍历图像每个像素点，将其灰度值作为索引，使该索引对应的数组元素加 1 即可。为了在窗口中显示，需将统计结果进行归一化处理，并映射到固定的画布高度上绘制线条。

2.2 傅里叶变换与卷积定理（任务三）

根据信号与系统理论中的卷积定理，空间域的卷积操作等价于频率域的逐元素相乘操作：

$$f(x, y) * h(x, y) \iff \mathcal{F}\{f\} \cdot \mathcal{F}\{h\} \quad (2)$$

当滤波器核 $h(x, y)$ 的尺寸较大（如本实验中的 128×128 高斯核）时，直接在空间域进行二维滑窗卷积的计算复杂度极高。通过快速傅里叶变换（FFT）将图像和核转换到频域，进行点乘后再使用逆傅里叶变换（IFFT）还原到空间域，可以显著降低计算耗时。为避免频域相乘带来的“循环卷积”引起的边缘缠绕效应，需在变换前对图像和卷积核进行零填充（Zero Padding），扩展至 $M_{img} + M_{kernel} - 1$ 的最优尺寸。

3 实验步骤

1. **环境配置：**配置 C++ 开发环境及 OpenCV 库，确保能够正常读取和显示图像。
2. **直方图（任务二）实现：**
 - 读取彩色图像并转化为单通道灰度图。
 - 定义大小为 256 的整型数组并清零。双重循环遍历图像 ‘rows’ 和 ‘cols’，统计各灰度级像素数。
 - 找出数组中的最大值，按比例将数组元素映射为高度，利用 ‘cv::line’ 函数在白色背景矩阵上绘制黑色直线，最终输出显示。

3. FFT 卷积（任务三）实现：

- 读取灰度图并转换为 ‘CV_32F’ 浮点型。
- 利用 ‘getGaussianKernel’ 生成一维高斯核（大小 128， $\sigma = 20$ ），通过矩阵乘法生成二维核。
- 利用 ‘getOptimalDFTSize’ 获取最优扩展尺寸，使用 ‘copyMakeBorder’ 对图像和核进行右下角的补零扩展。
- 将扩展后的图像和核组合为双通道复数矩阵，使用 ‘dft’ 函数进行正向傅里叶变换。
- 使用 ‘mulSpectrums’ 函数对频域复数矩阵进行逐元素相乘。
- 调用 ‘dft’ 进行逆变换（包含 ‘DFT_INVERSE | DFT_SCALE’ 参数），分离实部并按原核大小计算中心偏移量，裁剪出 ‘same’ 模式的有效图像区域。
- 转换为 ‘uint8’ 格式，记录程序运行时间并输出结果。

4 程序代码

注：根据实验要求，代码中已统一使用相对路径读取图片。

4.1 任务二：直方图 + 显示 (C++)

Listing 1: task2.cpp - 灰度直方图计算与显示

```
#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <stdio.h>
using namespace cv;
using namespace std;

Mat myHist(Mat img)
{
    Mat Hist;
    int H[256]={0};
    for(int m=0; m<img.rows; m++){
        for(int n=0; n<img.cols; n++){
            int val = img.at<uchar>(m, n);
            H[val] = H[val] + 1;
        }
    }
    int max_val = 0;
    for (int i = 0; i < 256; i++) {
        if (H[i] > max_val) {
            max_val = H[i];
        }
    }

    int hist_w = 512;
```

```

int hist_h = 400;
int bin_w = cvRound((double)hist_w / 256);
Hist = Mat(hist_h, hist_w, CV_8UC3, Scalar(255, 255, 255));

for (int i = 0; i < 256; i++) {
    int intensity = cvRound((double)H[i] / max_val * hist_h);
    line(Hist,
        Point(bin_w * i, hist_h),
        Point(bin_w * i, hist_h - intensity),
        Scalar(0, 0, 0),
        2, 8, 0);
}
return Hist;
}

int main()
{
    Mat input = imread("testing.jpg");
    if(input.empty()) {
        cout << "Error: Image not found!" << endl;
        return -1;
    }
    Mat gray;
    cvtColor(input, gray, COLOR_BGR2GRAY);
    Mat Hist = myHist(gray);

    imshow("Hist", Hist);
    waitKey(0);
    return 0;
}

```

4.2 任务三：卷积 (C++)

Listing 2: task3.cpp - 基于频域 FFT 的图像卷积计算

```

#include <iostream>
#include <fstream>
#include "opencv2/opencv.hpp"
#include "opencv2/imgproc/imgproc.hpp"
#include "opencv2/highgui/highgui.hpp"
#include <stdio.h>

using namespace cv;
using namespace std;

Mat myConv(Mat img)
{
    Mat Conv_img;
    // 开始计时

```

```

double t = (double)getTickCount();

// 生成高斯滤波器
int ksize = 128;
double sigma = 20.0;
Mat k1D = getGaussianKernel(ksize, sigma, CV_32F);
Mat h = k1D * k1D.t();

// 图像数据格式转换
Mat img_float;
img.convertTo(img_float, CV_32F);

// 扩充图像和滤波器的尺寸
int dft_rows = getOptimalDFTSize(img.rows + h.rows - 1);
int dft_cols = getOptimalDFTSize(img.cols + h.cols - 1);
Mat padded_img, padded_h;
copyMakeBorder(img_float, padded_img, 0, dft_rows - img.rows, 0, dft_cols - img.cols,
    BORDER_CONSTANT, Scalar::all(0));
copyMakeBorder(h, padded_h, 0, dft_rows - h.rows, 0, dft_cols - h.cols, BORDER_CONSTANT,
    Scalar::all(0));

// 傅里叶变换
Mat planes_img[] = { padded_img, Mat::zeros(padded_img.size(), CV_32F) };
Mat complex_img;
merge(planes_img, 2, complex_img);
dft(complex_img, complex_img);

Mat planes_h[] = { padded_h, Mat::zeros(padded_h.size(), CV_32F) };
Mat complex_h;
merge(planes_h, 2, complex_h);
dft(complex_h, complex_h);

// 频域乘法
Mat complex_res;
mulSpectrums(complex_img, complex_h, complex_res, 0);

// 逆傅里叶变换
dft(complex_res, complex_res, DFT_INVERSE | DFT_SCALE);
Mat planes_res[2];
split(complex_res, planes_res);
Mat result = planes_res[0]; // 提取实部作为结果

// 裁剪与格式转换
int start_x = (h.cols - 1) / 2;
int start_y = (h.rows - 1) / 2;
Rect roi(start_x, start_y, img.cols, img.rows);
Mat same_result = result(roi);
same_result.convertTo(Conv_img, CV_8UC1);

```

```

t = ((double)getTickCount() - t) / getTickFrequency();
cout << "基于FFT的图像卷积耗时: " << t << " 秒。" << endl;

return Conv_img;
}

int main()
{
    Mat input = imread("testing.jpg");
    if (input.empty()) {
        cout << "无法读取图像 testing.jpg, 请检查图片路径" << endl;
        return -1;
    }

    Mat gray;
    cvtColor(input, gray, COLOR_BGR2GRAY);
    Mat Conv_img = myConv(gray);

    imshow("Conv_img", Conv_img);
    waitKey(0);
    return 0;
}

```

5 实验结果显示

5.1 任务二：直方图显示

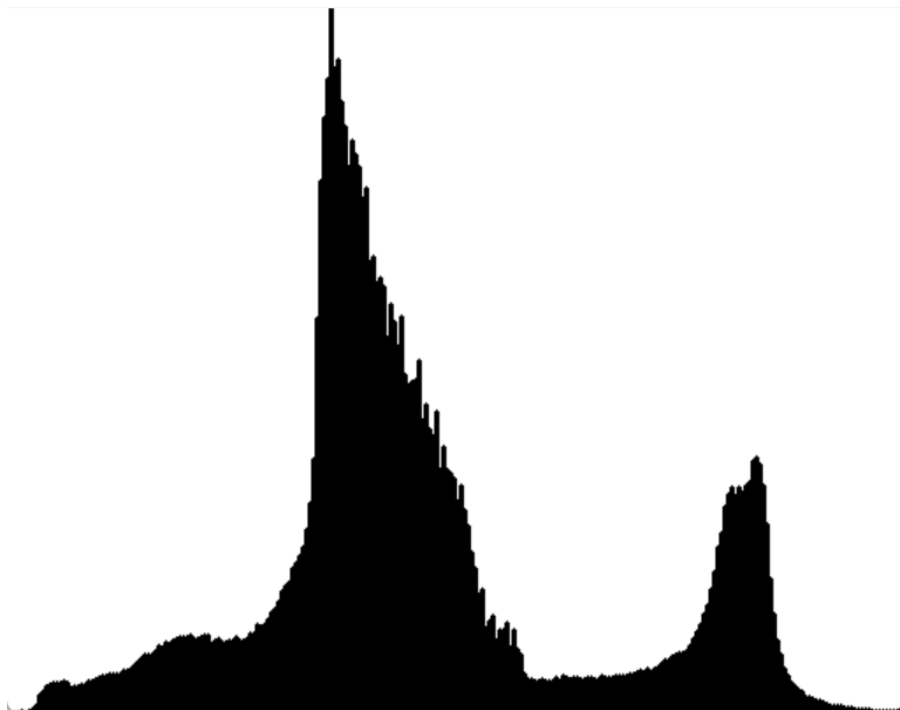


图 1: Task2 直方图显示结果

在运行任务二的代码后，程序会遍历原图的所有像素点，统计出 0-255 各灰度值的分布情况，并归一化后以柱状图/折线图的形式渲染在白底黑线的窗口中。

5.2 任务三：频域卷积显示与耗时

控制台输出信息如下：

基于 FFT 的图像卷积耗时：0.0753593 秒。



图 2: Task3 结果

6 实验分析总结

- 图像直方图的物理意义：**直方图直观地展示了图像亮度的分布情况，通过手写代码统计像素极大地加深了对图像作为“二维矩阵”本质的理解。在寻找最大值时，为了防止在绘制界面时溢出，归一化映射逻辑（`H[i] / max_val * hist_h`）是非常必要的。
- FFT 卷积的高效性：**本实验使用了尺寸高达 128×128 、 $\sigma = 20$ 的高斯滤波器。如果直接在空域使用嵌套循环进行滤波处理，时间复杂度将达到 $\mathcal{O}(M \cdot N \cdot K^2)$ ，处理耗时会极长。通过将其转换至频域处理，时间复杂度降低至 $\mathcal{O}(M \cdot N \log(M \cdot N))$ 。实验结果显示，如此巨大尺度的卷积仅耗时 **0.075 秒**左右，充分体现了 FFT 在大型矩阵卷积中的性能优势。
- 边界效应与黑边产生原因：**观察结果可以发现，图像表现出极强的模糊效果（证明了巨大高斯核发挥了低通滤波器的作用）。同时图像四周出现了渐变的黑色边缘（Black Borders）。这是因为在进行傅里叶变换前，对图像进行了‘copyMakeBorder’补零操作。在频率相乘后进行逆变换时，原有的像素数据与补入的零边界混合，使得边缘区域整体灰度值被拉低。截取中心（‘same’模式）后，边缘发黑属于数学卷积原理下的正常物理现象。